VIETNAM NATIONAL UNIVERSITY OF HOCHIMINH CITY
THE INTERNATIONAL UNIVERSITY
SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

NORMAL FIELD DIFFUSION FOR GEOMETRY-AWARE HOLE FILLING IN 3D TRIANGLE MESHES: A MESHLAB PLUGIN IMPLEMENTATION

By
Truong Tri Dung
MITIU25208

A project report submitted to the School of Computer Science and Engineering
in partial fulfillment of the requirements for the course
Advanced Computer Graphics

Ho Chi Minh City, Vietnam
Year 2026

TABLE OF CONTENTS

TABLE OF CONTENTS 2

LIST OF FIGURES 4

LIST OF TABLES 5

ABSTRACT 6

1 INTRODUCTION 7
- 1.1 Background . . . 7
- 1.2 Problem and scope . . . 8
- 1.3 Outline . . . 8

2 RELATED WORK 9
- 2.1 Combinatorial fillers . . . 9
- 2.2 Variational and PDE methods . . . 10
- 2.3 Normal-field and curvature-based methods . . . 11
- 2.4 Foundations . . . 11
- 2.5 Where NFD sits . . . 12

3 MATHEMATICAL PRELIMINARIES 13
- 3.1 Cotangent Laplacian . . . 13
- 3.2 Heat equation and implicit Euler . . . 14
- 3.3 Taubin smoothing . . . 14

4 METHODOLOGY 15
- 4.1 Pipeline overview . . . 15
- 4.2 Hole detection . . . 16
- 4.3 Boundary analysis . . . 16
- 4.4 Initial triangulation . . . 17
- 4.5 Normal field diffusion . . . 18
- 4.6 Laplacian smoothing . . . 20
- 4.7 Curvature-guided displacement . . . 20
- 4.8 Patch merge . . . 22

5 IMPLEMENTATION 23
- 5.1 Plugin architecture . . . 23

5.2 VCGlib usage 24
5.3 Eigen for the diffusion solver 24
5.4 Parameters and code layout 24
5.5 Build 25

6 EXPERIMENTAL RESULTS 26
6.1 Test set and metric 26
6.2 Hausdorff measurements 27
6.3 Per-model figures 28
6.4 Comparison with MeshLab Close Holes 30
6.5 Performance 30

7 DISCUSSION 32
7.1 Behaviour at default parameters 32
7.2 Failure modes 32
7.3 Limitations 33

8 CONCLUSION 34

REFERENCES 35

APPENDIX 37

LIST OF FIGURES

1.1 Input mesh with a hole (left) and the NFD result (right). 7

3.1 Cotangent weight on edge (v_i, v_j), using the two opposite angles α_{ij} and β_{ij}. . . 13

4.1 The seven-stage pipeline. 15
4.2 Projection of the 3D boundary (blue) onto the tangent plane defined by $\bar{\mathbf{n}}$. . . . 17
4.3 FIFO ear clipping (left) versus best-ear clipping (right) on the same polygon. . 17
4.4 Edge flip replaces diagonal (u, v) with (w, x) when this increases the minimum angle. 18
4.5 Heat-equation diffusion of the normal field, from initial state to harmonic steady state. 19
4.6 Spherical-cap geometry: chord radius r, half-angle $\theta/2$, cap height $h = r\tan(\theta/2)$. 20
4.7 Normalised geodesic distance t on a triangulated patch. 21
4.8 Displacement weighting profiles. 22
4.9 Taubin smoothing removes high-frequency spikes (left) without flattening the dome (right). 22

5.1 Filter parameter dialog generated from plugin metadata. 23
5.2 MeshLab log output after a successful filter invocation. 25

6.1 `sphere_small`: input (left) and NFD result (right). 28
6.2 `sphere_large`: input (left) and NFD result (right). 28
6.3 `cylinder_side`: input (left) and NFD result (right). 28
6.4 `torus`: input (left) and NFD result (right). 29
6.5 Stanford `bunny`: six holes (left) all filled (right). 29
6.6 `cow`: input (left) and NFD result (right). 29
6.7 NFD (left) versus MeshLab `Close Holes` (right) on the same input. 30
6.8 Ear clipping without (left) and with (right) Delaunay edge flipping. 30

7.1 Overshoot with `CurvatureStrength = 1.5` on a sharp Bunny hole. 32

LIST OF TABLES

5.1 Plugin parameters. 25

6.1 Test models. 26
6.2 Hausdorff results at default parameters. RMS is forward direction; percentages are relative to the BBox diagonal. 27
6.3 Single-threaded timings. 30

ABSTRACT

Triangle meshes from scan or reconstruction pipelines often contain holes, and most existing fillers produce visibly flat patches because they triangulate the boundary in a plane. This report describes a MeshLab plugin implementing *Normal Field Diffusion* (NFD), which gives the patch its curvature in two stages. The boundary vertex normals are treated as Dirichlet data and diffused across the patch interior using the heat equation. Each interior vertex is then displaced along its diffused normal by an amount derived from a spherical-cap fit to the boundary. Implicit Euler with a one-time Cholesky factorisation keeps the inner solver stable and inexpensive. On six ground-truth meshes (spheres, cylinder, torus, Stanford Bunny, cow), the symmetric Hausdorff error reaches 0.12% on the simplest case and is roughly 2–4$\times$ smaller than MeshLab's built-in `Close Holes` filter on curved surfaces.

Keywords: hole filling, mesh repair, cotangent Laplacian, heat equation, MeshLab.

CHAPTER 1

INTRODUCTION

1.1. Background

Mesh data from scanners or photogrammetry is rarely watertight. Self-occlusion, low-confidence regions, glossy surfaces, and post-process editing all introduce holes. The presence of a hole is enough to break rendering (the back side becomes visible through it), most physical simulations (which assume a closed domain), and 3D-print slicers.

Standard hole fillers, such as ear clipping or the advancing-front method behind MeshLab's `Close Holes`, close the topological gap but produce a patch that is essentially flat. On a sphere this is immediately visible as a dimple. More sophisticated alternatives exist — Poisson reconstruction, thin-plate splines — but they either reshape the entire mesh or require solving a fourth-order PDE, neither of which fits an interactive plugin model.

The hole-filling problem appeared on the Advanced Computer Graphics project list, but only the problem itself was specified, not the approach. NFD was chosen after a short literature survey: the Verdera et al. paper on inpainting surface holes offered the most direct route from boundary normals to a curved patch, and most of its machinery (cotangent Laplacian, heat equation, sparse linear solvers) was already covered in the course.

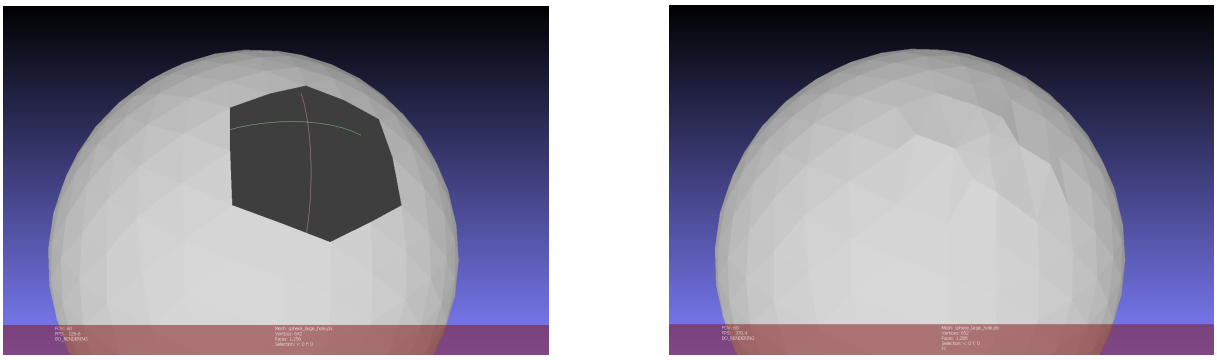

Figure 1.1: Input mesh with a hole (left) and the NFD result (right).

Normal Field Diffusion (NFD) sits between these extremes. It triangulates the hole as a flat patch first, then diffuses the boundary normals across the patch interior using a discrete heat equation, and finally displaces each interior vertex along its diffused normal by an amount estimated from the boundary geometry. The patch ends up curved, matches the surrounding surface, and the rest of the mesh is left alone.

1.2. Problem and scope

Given a mesh $\mathcal{M} = (V, F)$ with one or more boundary loops, the goal is to add vertices and faces so that each loop becomes interior, the patch joins the surrounding surface continuously, and the new triangles are roughly the same size and shape as their neighbours. The topological closure is trivial. Triangle quality is handled by standard heuristics. The hard part is geometric continuity, which is what this report focuses on.

The implementation is a self-contained MeshLab plugin in C++ on top of VCGlib and Eigen. It processes one hole at a time, leaves everything else untouched, and exposes seven parameters in the standard filter dialog. Evaluation uses MeshLab's built-in Hausdorff sampler against ground-truth meshes generated from the same source models.

1.3. Outline

Chapter 2 surveys related work and places NFD in context. Chapter 3 collects the mathematical background — discrete Laplace–Beltrami, the heat equation, implicit Euler, Taubin smoothing. Chapter 4 walks through the seven pipeline stages. Chapter 5 covers the plugin implementation. Chapter 6 reports the Hausdorff measurements and compares NFD with MeshLab's `Close Holes` filter. Chapters 7 and 8 discuss limitations and conclude.

CHAPTER 2

RELATED WORK

Hole filling on triangle meshes has a long history. The papers that turned out to matter for this project fall into three rough groups: combinatorial methods that triangulate the hole as a polygon, variational methods that solve a global energy on the patch, and normal-field methods that propagate orientation information from the boundary inward. NFD borrows from all three and is closest in spirit to the third.

2.1. Combinatorial fillers

The ear-clipping family is the oldest. Meisters [12] proved in 1975 that every simple polygon has at least two ears, which is the topological fact that makes ear clipping work. Held's FIST system [7] is the robust industrial implementation, and most mesh-tool ear clippers descend from it either directly or by reimplementation. The version in this report borrows the layered fallback structure from FIST without using its exact-arithmetic predicates, because the patch sizes encountered here are small enough that ordinary floating-point comparisons do not cause trouble.

For 3D hole filling the canonical reference is Liepa [11], an advancing-front method that picks each new triangle directly in 3D rather than projecting to 2D and lifting back. Liepa is what MeshLab's `Close Holes` filter implements, and it is the direct baseline this report compares against in Chapter 6. The patch it produces is geometrically flat in the following sense: each new triangle is chosen on local quality criteria (dihedral angle, edge length, balance of the front) without any input from the curvature of the surrounding mesh. On a sphere this is immediately visible.

Barequet and Sharir [2] formulate the same problem as a constrained Delaunay triangulation, which is theoretically cleaner but expensive and not native to the MeshLab/VCGlib tooling. The standard approximation is Lawson-flip iteration, which locally improves quality without solving the global problem. The implementation here applies it twice, once after initial ear clipping and once after centroid refinement.

The combinatorial line shares one structural limit. The patch is fully determined by the boundary polygon and the heuristic, so it cannot reflect curvature information from outside the polygon — there is no mechanism for that to enter. The next two groups of work address this in different ways.

2.2. Variational and PDE methods

The simplest variational approach treats the patch as a discrete minimum surface and solves $\Delta u = 0$ on patch positions with the boundary fixed. It is fast, gives a smooth patch, and biases toward flatness because it minimises area. On a sphere the patch sits inside the chord rather than on the sphere; the dimple problem of ear clipping is reproduced through different machinery. Botsch and Kobbelt's Laplacian deformation framework [4], although the paper itself is about interactive editing rather than hole filling, is built on the same Laplacian-with-Dirichlet machinery and shares the same shrinkage tendency when adapted to hole filling.

Going to a fourth-order energy — bi-Laplacian, equivalent to thin-plate splines — lets the patch encode boundary tangents in addition to positions, and produces C^2 patches that can bulge outward when the surrounding surface is curved. Nealen et al. [14] present a unified Laplacian/bi-Laplacian framework that covers this as one of several modes. The cost is a denser, worse-conditioned linear system: the bandwidth of Δ^2 is roughly the square of Δ on a triangle mesh, and direct solvers slow down accordingly. For a plugin that runs inside MeshLab's interactive loop this lives on the wrong side of the speed/quality trade-off.

Of all the references in this section, the closest in spirit to NFD is Pernot, Moraru, and Véron [15]. Instead of minimising a quadratic energy on positions, they minimise the variation of curvature across the patch, simulated as a thin-plate mechanical model whose boundary forces are driven by the surrounding curvature. The high-level intuition — carry curvature information from the boundary inward — is the same as NFD's, but the formulation is on positions, the iteration is non-linear, and the input must be smoothly parameterisable. Reading their paper alongside Verdera et al. helped clarify what is actually being asked of the patch: not just a smooth surface, but one whose curvature continues that of the boundary.

Poisson surface reconstruction [9, 10] occupies a different niche. It is not strictly a hole filler — the input is a point cloud or oriented mesh and the output is a globally smooth surface, with holes filled as a side effect. For hole-repair purposes the difficulty is exactly that. Parts of the input that did not need fixing get reshaped along with the broken parts, and the input-output triangle correspondence is lost in the volumetric intermediate.

Laplacian Surface Editing by Sorkine et al. [18] is mentioned here because of a useful conceptual distinction it sharpens: the Laplacian can be used as an *operator* on positions or as a *representation* of geometry through differential coordinates. Minimum-surface and bi-Laplacian hole fillers use the Laplacian as an operator. Differential-coordinates methods use it as a representation. NFD belongs to the first camp, although its operator is applied to normals rather than positions.

2.3. Normal-field and curvature-based methods

Verdera, Caselles, Bertalmío, and Sapiro [20] is the closest precursor and where the core idea of NFD originates. They diffuse each component of the surface normal field across the hole through a non-linear PDE, with boundary normals as Dirichlet data, and reconstruct positions afterwards. NFD is essentially this idea translated from level-set surfaces (where they discretise the PDE on a regular volumetric grid) to triangle meshes (where the cotangent Laplacian gives the right discrete operator directly). The level-set framework was a reasonable choice in 2003 because the triangle-mesh discrete-differential-geometry tooling was still settling: Pinkall–Polthier was known but Meyer et al. had only just appeared, and not every implementation used cotangent weights as a default. Translating to triangle meshes today is more or less mechanical.

The diffusion idea itself comes from earlier work on image inpainting by Bertalmío, Sapiro, Caselles, and Ballester [3], from the same research group. Verdera et al. is the surface-normals analogue of Bertalmío et al.'s pixel-intensities approach. Knowing this connection was useful while reading the surface-holes paper, because the level-set notation is dense and the underlying intuition is clearer when read alongside the 2D image case.

Local normal-interpolation methods are a parallel branch that interpolates boundary normals into the interior without a global PDE. Wang and Oliveira [21] use radial basis functions, Jun [8] uses a moving-least-squares approach with sub-region partitioning. Without a global smoothness criterion these methods are simpler to implement but degrade on larger or non-convex holes; the seams between sub-regions in Jun's piecewise approach in particular tend to need separate cleanup.

Zhao, Gao, and Lin [23] sit between Verdera et al. and the local methods. They use a Poisson-style PDE for fairing patch positions, combined with a separate boundary-driven displacement step. The two-stage structure is the same as Stages 4 and 6 in this report, but the variable they solve for is position, not normal, so the formulation inherits the shrinkage of position-based methods — partially compensated by the displacement step but not eliminated.

2.4. Foundations

The discrete operators NFD relies on come from the broader discrete-differential-geometry literature. The cotangent Laplacian was introduced by Pinkall and Polthier [16] in 1993 for discrete minimal-surface construction, and was packaged as a standard geometry-processing operator by Meyer et al. [13] a decade later, with the mean-curvature normal identity that Stage 2 of NFD uses. Wardetzky et al. [22] gave the useful negative result that no discrete Laplacian satisfies all desirable properties simultaneously, which retroactively explains why several different cotangent-style operators coexist in the literature. The version used here is convergent and intrinsic but breaks the maximum principle on obtuse triangles, and NFD handles this by clamping negative weights to zero. Sharp and Crane [17] address the same problem more cleanly through

intrinsic Delaunay remeshing; that would be a worthwhile upgrade for inputs with many obtuse triangles.

Two other operators are reused as-is. Desbrun et al. [6] introduced implicit fairing by diffusion in 1999, which is essentially the numerical recipe NFD uses in Stage 4 (assemble the cotangent Laplacian, solve the implicit-Euler system, iterate) but applied to vertex positions rather than normals. Taubin's (λ, μ) filter [19] is the standard non-shrinking low-pass smoother, used in Stage 6 of NFD because uniform Laplacian smoothing would flatten the dome. Crane et al. [5] proposed the heat method for geodesic distance computation, which would be a more accurate replacement for the multi-source Dijkstra in Stage 6; we have not measured how much accuracy it would buy in practice on the test patches.

The mesh-repair survey of Attene, Campen, and Kobbelt [1] is the right reference for context. Hole filling is one of many mesh-defect classes; a production-grade repair tool needs to handle the others (non-manifold edges, self-intersections, inverted normals) as well. NFD assumes the input has been cleaned of those defects, which is reasonable inside MeshLab where standard cleanup filters are one menu away.

2.5. Where NFD sits

The components of NFD are individually well-known: heat-equation diffusion of normals is from Verdera et al., the cotangent Laplacian is standard, Taubin smoothing dates from 1995. What this report contributes is putting them together in a way that runs end-to-end inside MeshLab without voxelisation, with an implicit-Euler solver and a one-time LDL^T factorisation that keeps the inner loop fast, a spherical-cap displacement model whose dome height is derived from boundary geometry rather than supplied as a parameter, and a $\sqrt{2t - t^2}$ profile that gives tangent-plane continuity at the patch boundary — the property the more common parabolic profile fails to provide.

Among the closest references — Verdera et al., Pernot et al., Zhao et al. — none combine these specific choices in the same way. The result is best understood as a practical translation of Verdera-style normal diffusion into the triangle-mesh setting, picked apart into a sequence of stages that each have a clear role and a known fallback when they fail.

CHAPTER 3

MATHEMATICAL PRELIMINARIES

This chapter collects the four pieces of background needed to read Chapter 4: the cotangent Laplacian, the heat equation on a manifold, implicit Euler time stepping, and Taubin smoothing.

3.1. Cotangent Laplacian

A triangle mesh is a pair $\mathcal{M} = (V, F)$ with vertex set $V \subset \mathbb{R}^3$ and triangular face set F. An edge is a *border* edge if it belongs to exactly one face; a closed chain of border edges is a hole.

The Laplace–Beltrami operator $\Delta_{\mathcal{S}}$ on a smooth surface admits several discrete versions. The simplest is the uniform Laplacian, $L_u(v_i) = \sum_j (v_j - v_i)/d_i$, which is cheap but mesh-density dependent. The cotangent Laplacian [16, 13] weights each neighbour by

$$w_{ij} = \cot \alpha_{ij} + \cot \beta_{ij}, \tag{3.1}$$

where α_{ij} and β_{ij} are the two angles opposite the edge (v_i, v_j) in the two faces sharing it (Figure 3.1). The cotangent Laplacian converges to $\Delta_{\mathcal{S}}$ under refinement, is invariant under intrinsic remeshing, and gives the standard mean-curvature normal $L_{\text{cot}}(v_i) = -2H_i\mathbf{n}_i$. We use it everywhere in this report.

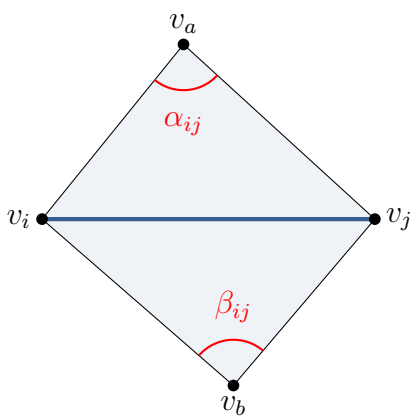

Figure 3.1: Cotangent weight on edge (v_i, v_j), using the two opposite angles α_{ij} and β_{ij}.

A practical detail: $\cot \theta < 0$ for obtuse angles, which can break the maximum principle of the discrete Laplacian. We clamp $w_{ij} \leftarrow \max(w_{ij}, 0)$, which is simple and works on the test meshes. The cleaner alternative is intrinsic Delaunay remeshing [17], but that is substantially more code.

3.2. Heat equation and implicit Euler

The heat equation on a domain Ω with Dirichlet boundary data g is

$$\frac{\partial u}{\partial t} = \lambda \Delta_\Omega u, \qquad u\Big|_{\partial\Omega} = g. \tag{3.2}$$

As $t \to \infty$, the solution converges to the harmonic extension of g into Ω — the smoothest interpolation in the sense of Dirichlet energy. This is the property we want for the diffused normal field on the patch.

For time stepping there is a choice between explicit and implicit Euler. Explicit Euler is conditionally stable, requiring $\Delta t < 2/\rho(L)$, which gets unfavourable as the mesh refines. Implicit Euler,

$$(I + \lambda L)\, u^{t+1} = u^t, \tag{3.3}$$

is unconditionally stable for any $\lambda > 0$ when L is positive semi-definite. The matrix $I + \lambda L$ is symmetric positive definite, sparse, and stays the same across iterations — only the right-hand side changes. This is exactly the case where a one-time Cholesky factorisation pays off.

We use `Eigen::SimplicialLDLT`, which factors the matrix as $A = LDL^T$ once and then solves each iteration in $O(\text{nnz}(A))$. For the patch sizes we encounter (interior vertex counts of order 10^2–10^3), a single factorisation amortised over 50 iterations is essentially free.

3.3. Taubin smoothing

Plain Laplacian smoothing $v \leftarrow v + \lambda L(v)$ moves each vertex toward the centroid of its neighbours and works well for cleaning high-frequency noise — but iterated, it shrinks the surface globally. Taubin [19] fixes this with a two-step pass: a positive shrink step ($\lambda > 0$) followed by a negative inflate step ($\mu < 0$, $|\mu| > \lambda$). The combined transfer function in the Laplacian eigenbasis,

$$f(k) = (1 - \lambda k)(1 - \mu k), \tag{3.4}$$

satisfies $f(0) = 1$ (DC preserved) and $|f(k)| < 1$ for high frequencies (noise attenuated). The result is a low-pass filter without shrinkage. We use $\lambda = 0.40$, $\mu = -0.44$, three pairs — a standard choice from the literature.

CHAPTER 4

METHODOLOGY

4.1. Pipeline overview

The plugin runs seven stages per detected hole. Stage 4 is the algorithmic core; the others prepare the patch for it or clean up after it. Stages 5 and 6 must run in this order: uniform Laplacian smoothing applied after displacement would erase the dome that displacement creates, because uniform smoothing exhibits shrinkage on curved surfaces.

This ordering was reversed in an early version of the implementation. Stage 6 produced patches with localised spike artifacts, and applying Stage 5 afterwards seemed like a natural cleanup pass — right up until the result on `sphere_large` was overlaid on the ground truth and the dome turned out to have been quietly flattened. Reordering the two stages and using Taubin (which preserves low frequencies) in Stage 6 for spike cleanup fixed both problems at once.

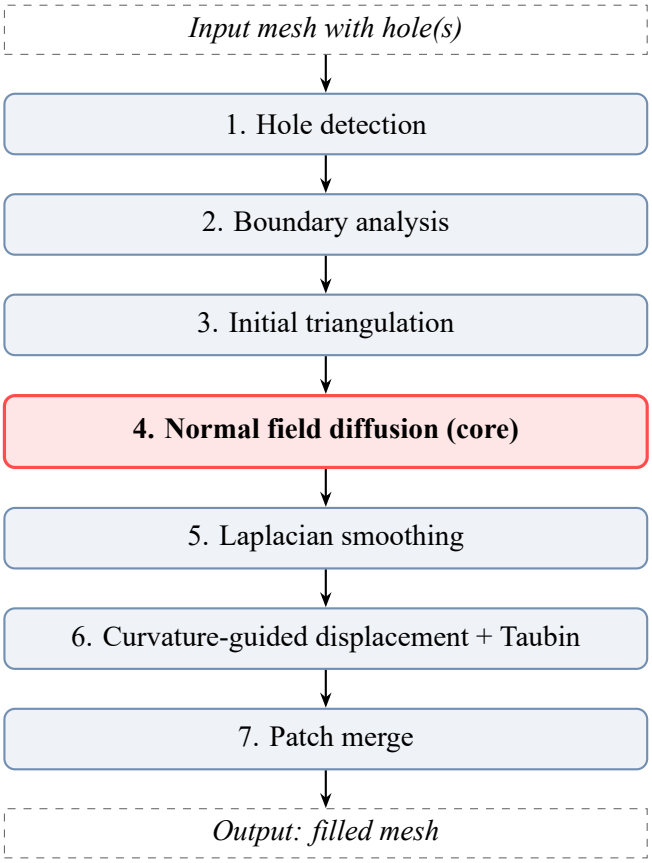

Figure 4.1: The seven-stage pipeline.

4.2. Hole detection

VCGlib's adjacency machinery does most of the work. After enabling face–face and vertex–face adjacency and calling `UpdateFlags::FaceBorderFromFF`, each border edge has its flag set, and we walk the boundary by hopping from one border edge to the next using `VFIterator`. In a 2-manifold mesh each boundary vertex has exactly two incident border edges, so excluding the previous vertex uniquely determines the next one.

Algorithm 1 Boundary loop walk

```
for each face f, each local edge e do
    if IsBorder(f, e) and edge unvisited then
        start ← V_e; curr ← V_e; next ← V_{e+1}
        while curr ≠ start on subsequent iterations do
            append curr; mark (curr, next) visited
            prev ← curr; curr ← next
            find next border edge (curr, x) with x ≠ prev
            next ← x
        end while
    end if
end for
```

A safety counter and a length sanity check on the loop guard against non-manifold input. Holes longer than `MaxHoleSize` edges are skipped so the user can opt in.

4.3. Boundary analysis

For each boundary vertex, the area-weighted normal is read from `vp->N()` (cached by VCG after the topology update) and the mean curvature is computed from the cotangent Laplacian. The cotangent of an angle is evaluated as

$$\cot\theta = \frac{\vec{AB}\cdot\vec{AC}}{\|\vec{AB}\times\vec{AC}\|}, \tag{4.1}$$

which avoids calls to `acos` and `sin` and remains numerically stable near 0 and π. Weights accumulate face by face: in face (v_i, v_B, v_C), the angle at C contributes to w_{iB} and the angle at B contributes to w_{iC}. Each interior edge collects contributions from both adjacent faces; border edges receive only one, which matches the formula on an open surface.

The resulting mean curvature $H_i = \|L_{\text{cot}}\|/(2\sum_j w_{ij})$ is stored but not used downstream. The displacement step in Stage 6 estimates dome height from boundary normal angles directly, so H_i is kept only as a sanity-check value during debugging.

4.4. Initial triangulation

The patch is built in four sub-steps: project to 2D, ear-clip, Delaunay-flip, refine.

2D projection

Compute the average boundary normal $\bar{\mathbf{n}}$ and pick two perpendicular axes $(\mathbf{u}, \mathbf{v})$ to form a tangent frame. Project each boundary vertex by subtracting the centroid and dotting with $\mathbf{u}, \mathbf{v}$. The signed area via the shoelace formula tells us whether the projected polygon is CCW; if not, we reverse it.

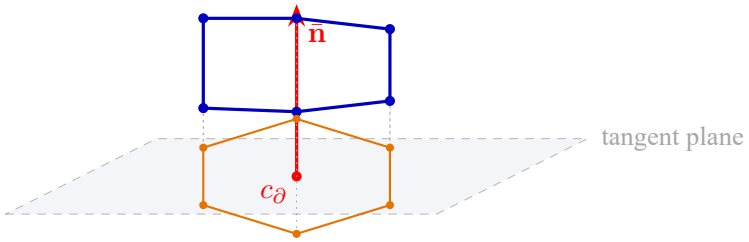

Figure 4.2: Projection of the 3D boundary (blue) onto the tangent plane defined by $\bar{\mathbf{n}}$.

Ear clipping

The naive version of ear clipping picks the first valid ear at each step, which on a regular polygon degenerates into a fan of sliver triangles (Figure 4.3, left). The implementation scans every valid ear at each step and picks the one whose minimum interior angle is largest. This produces a noticeably more even tessellation. Three fallback layers handle pathological input: if no valid ear exists, the first convex vertex is clipped; if no convex vertex exists, vertex 1 is force-clipped; if the polygon collapses below three vertices, the loop stops. A winding check against $\bar{\mathbf{n}}$ at the end flips the triangle ordering if the patch ended up facing the wrong way.

The first version of this stage used FIFO ear selection. The patch on `sphere_small` came out as a fan of slivers concentrated at one vertex, and downstream this caused Stage 4 to produce visibly biased normals near that vertex. Switching the heuristic to max-min interior angle was a five-line change and removed the bias.

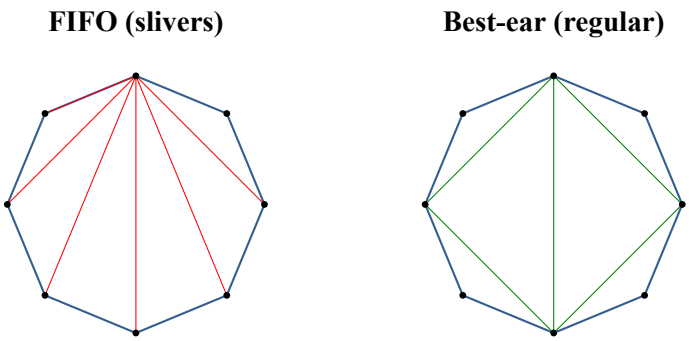

Figure 4.3: FIFO ear clipping (left) versus best-ear clipping (right) on the same polygon.

Delaunay flips and centroid refinement

For each interior edge shared by two triangles, the alternate diagonal is considered as a flip candidate. The flip is accepted only if it strictly increases the minimum interior angle across the pair. A face that has already been flipped in the current pass is locked until the next one, which prevents conflicting flips during a single sweep. On the test meshes, three passes are enough to converge.

The flip pass was the part of Stage 3 that took longest to get right. The issue was deriving the correct winding for the new diagonal on a curved 3D patch: a global sign-check against $\bar{\mathbf{n}}$ works for nearly planar patches but breaks once the patch curves enough that the new triangle faces the opposite side of $\bar{\mathbf{n}}$ from the original. After being stuck on this for an hour or two, going back to the lecture material on mesh orientation gave the right hint — the cyclic order of u, v inside the original triangle t_1 already encodes the orientation locally, so no global vector is needed. The two-case split in the code (whether t_1 contains $u \to v$ or $v \to u$) is the consequence.

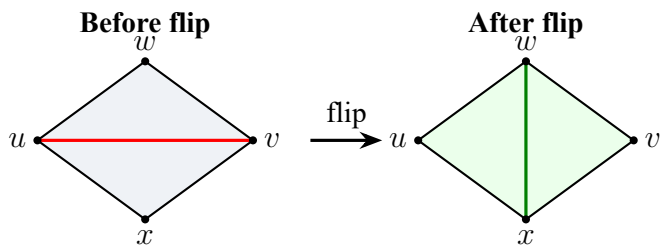

Figure 4.4: Edge flip replaces diagonal (u, v) with (w, x) when this increases the minimum angle.

The target triangle area is set to that of an equilateral triangle with side $\bar{e} \cdot \texttt{RefinementFactor}$, where $\bar{e}$ is the average boundary edge length. Faces with area above $1.5\times$ the target are split by inserting their centroid as a new interior vertex. Up to eight refinement passes run, capped at 200 000 faces total to prevent runaway subdivision on aggressive parameter values. A second Delaunay flip pass cleans up slivers introduced by centroid insertion.

4.5. Normal field diffusion

This is the stage that gives the patch its curvature. Each component of the vertex normal is treated as a scalar field on the patch, the boundary values are pinned to those from Stage 2, and the heat equation is used to propagate boundary information inward. The continuous problem is

$$\frac{\partial \mathbf{n}}{\partial t} = \lambda \, \Delta_\Omega \mathbf{n}, \qquad \mathbf{n}\Big|_{\partial\Omega} = \mathbf{n}^{\text{fixed}}, \tag{4.2}$$

applied component-wise. The steady state is the harmonic extension of the boundary normals into the patch interior.

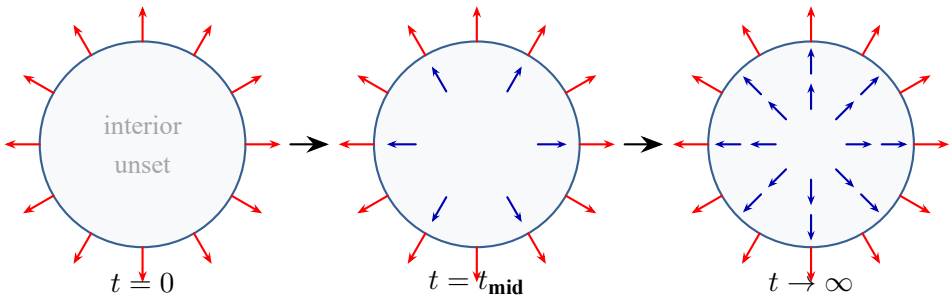

Figure 4.5: Heat-equation diffusion of the normal field, from initial state to harmonic steady state.

Discretising with implicit Euler gives, per component,

$$(I + \lambda L_{\text{cot}})\, \mathbf{n}_{\text{int}}^{t+1} = \mathbf{n}_{\text{int}}^{t} + \lambda\, \mathbf{b}, \tag{4.3}$$

where $\mathbf{b}_i = \sum_{j \in \partial \cap \mathcal{N}(i)} w_{ij}\, n_j^{\text{fixed}}$ collects the contributions from fixed boundary neighbours. This is the standard finite-element treatment of Dirichlet conditions: boundary values are eliminated from the unknown vector and reappear on the right-hand side. The system matrix is built from cotangent weights, computed face by face on the patch. Diagonal entries are $1 + \lambda \sum_j w_{ij}$, summing over *all* neighbours (both interior and boundary). Off-diagonal entries are $-\lambda w_{ij}$, limited to interior neighbours.

The matrix is symmetric positive definite, sparse, and constant across iterations, so it is factored once with `Eigen::SimplicialLDLT` and reused:

Algorithm 2 Iterative implicit Euler

```
A ← I + λ L_cot
solver.compute(A)                                  ▷ LDL^T factorisation, once
for t = 1 to DiffusionIterations do
    rhs ← n_int^t + λ b
    n_int^{t+1} ← solver.solve(rhs)
end for
Normalise each n_i
```

The default of 50 iterations is sufficient for visual convergence on the test meshes. After the loop, each diffused normal is normalised to unit length, with a guard for the rare zero-magnitude case. A separate NaN check in the caller catches any non-finite component (which can occur on extremely degenerate cotangent weights) and resets the interior normals to the boundary average.

4.6. Laplacian smoothing

Stage 5 redistributes the interior vertex *positions* on the still-flat patch using uniform Laplacian smoothing,

$$v_i^{(k+1)} = \frac{1}{|\mathcal{N}(v_i)|} \sum_{v_j \in \mathcal{N}(v_i)} v_j^{(k)}, \tag{4.4}$$

with boundary vertices held fixed. The uniform Laplacian is appropriate here because the goal is to even out vertex spacing on a flat surface, not to approximate a continuous operator. The default is three iterations.

4.7. Curvature-guided displacement

This stage bends the flat patch into a dome. The displacement amplitude at the dome apex is estimated from the boundary geometry by fitting an imaginary spherical cap. The cap radius is

$$r = \tfrac{1}{|\partial|} \sum_{i \in \partial} \|v_i - c_\partial\|, \tag{4.5}$$

the boundary's mean distance from its centroid, and the half-angle is

$$\theta = \arccos\left(\tfrac{1}{|\partial|} \sum_i \mathbf{n}_i \cdot \bar{\mathbf{n}}\right), \tag{4.6}$$

the mean angle between each boundary normal and the boundary's average normal. For a hole on a sphere this is exact: a chord of length r at half-angle $\theta/2$ from the cap apex gives cap height $h = r\tan(\theta/2)$. The user-facing `CurvatureStrength` parameter scales this:

$$h = r\ \tan(\theta/2) \cdot \texttt{CurvatureStrength}. \tag{4.7}$$

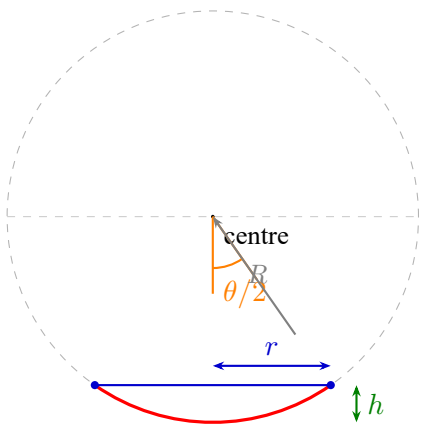

Figure 4.6: Spherical-cap geometry: chord radius r, half-angle $\theta/2$, cap height $h = r\tan(\theta/2)$.

The mean angle is used here, not the maximum. On the Bunny and the torus, a few boundary normals point in unusual directions, and using the maximum angle in those cases pushes the

dome too high. The mean turns out to be much more stable across the test set.

The first version of this stage used $\theta_{\max}$. It was correct on the spheres but on one of the Bunny's six holes, a noisy boundary normal pushed $\theta_{\max}$ to nearly $\pi/2$, and the patch in that hole came out as a tall spike rising well above the surrounding surface. Plotting the per-hole boundary-normal angle distribution made the failure mode obvious, and substituting the mean was a one-line change.

The per-vertex displacement weight depends on how far each interior vertex sits from the boundary. The distance is measured as a normalised geodesic distance via multi-source Dijkstra (Figure 4.7): $t_i = \text{dist}_i / \max_j \text{dist}_j$, so $t = 0$ at the boundary and $t = 1$ at the geodesically farthest interior vertex.

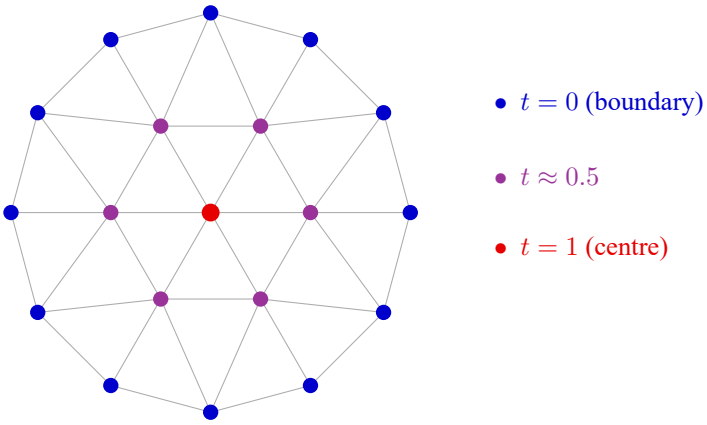

Figure 4.7: Normalised geodesic distance t on a triangulated patch.

The weighting profile is

$$w(t) = \sqrt{2t - t^2}, \qquad t \in [0, 1]. \tag{4.8}$$

Geometrically, this is the height of the unit hemisphere as a function of horizontal offset, which is consistent with the spherical-cap assumption. The more important property is $w'(0) = +\infty$: the tangent is vertical at $t = 0$, so the patch tangent at the boundary is parallel to the tangent plane of the surrounding mesh, giving approximate C^1 continuity. A parabolic profile $4t(1-t)$ has $w'(0) = 4$ and produces a visible crease where the patch meets the surface. Figure 4.8 compares both.

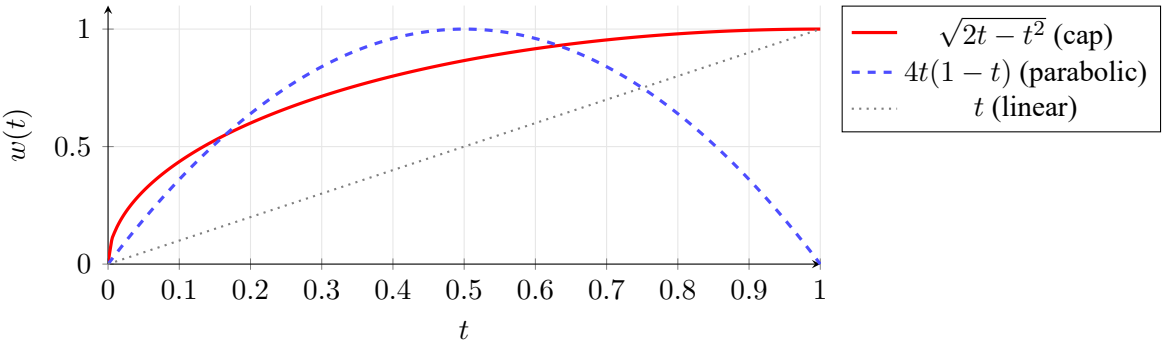

Figure 4.8: Displacement weighting profiles.

The actual displacement is

$$v_i' = v_i + h \cdot w(t_i) \cdot \mathbf{n}_i^{\text{diff}}, \tag{4.9}$$

where the direction is the diffused normal from Stage 4, not the constant $\bar{\mathbf{n}}$. Using the spatially varying field allows the patch to bend differently on different sides, which matters on saddle-shaped boundaries.

After displacement, a few interior vertices may carry spike-like noise from irregular triangulation. Three pairs of Taubin smoothing ($\lambda = 0.40, \mu = -0.44$) clean this up: each pair is a positive (shrink) Laplacian step followed by a stronger negative (inflate) step. The combined filter attenuates high frequencies while preserving the dome.

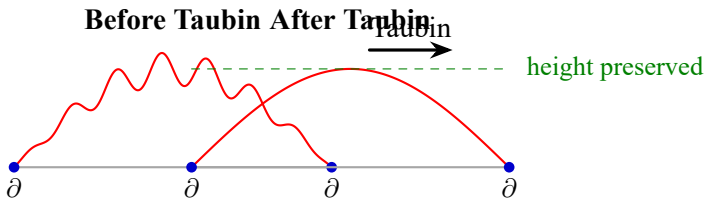

Figure 4.9: Taubin smoothing removes high-frequency spikes (left) without flattening the dome (right).

4.8. Patch merge

The final stage stitches the patch into the host mesh. Boundary patch vertices are remapped to their original indices in $\mathcal{M}$ to avoid duplication. Interior patch vertices are appended with `AddVertices`, and faces are added with `AddFaces` after index translation. Vertex and face colours are averaged from boundary neighbours, so the patch does not appear as a white stripe on a coloured mesh. A final round of `UpdateTopology`, `UpdateNormal`, and `UpdateBounding` restores cached adjacency and attributes.

CHAPTER 5

IMPLEMENTATION

5.1. Plugin architecture

MeshLab loads filters as dynamic libraries discovered at startup via Qt's `QPluginLoader`. A filter plugin inherits from `FilterPlugin`, is annotated with `Q_OBJECT` and `Q_PLUGIN_METADATA` so the Qt MOC can wire it up, and overrides a handful of virtual methods that report metadata, declare parameters, and execute the filter. The MeshLab UI is generated automatically from the parameter list, so the only UI code needed in the plugin is the `initParameterList` call that registers the seven user parameters with their types, defaults, and tooltips.

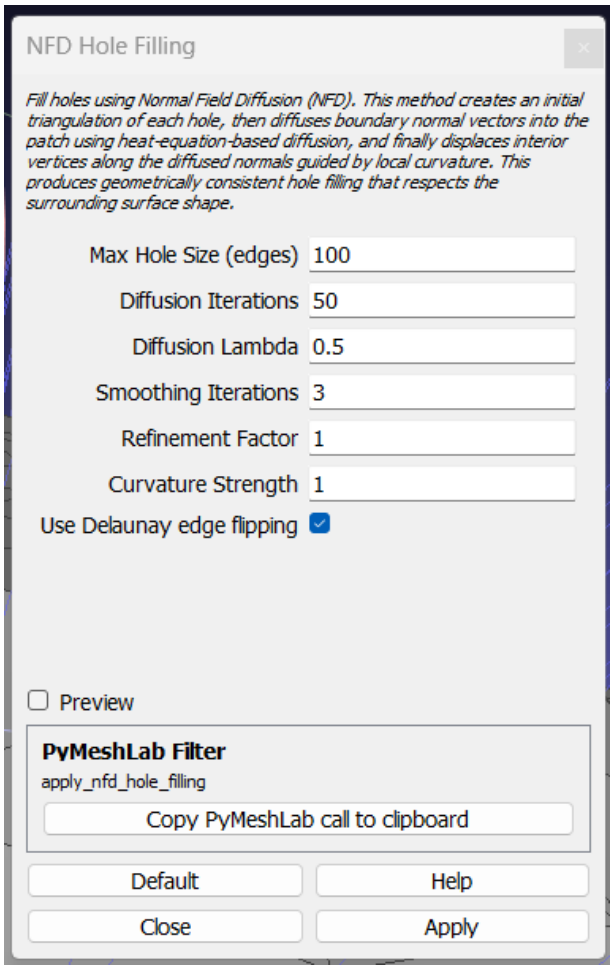

Figure 5.1: Filter parameter dialog generated from plugin metadata.

The plugin is built as a CMake `MODULE` target that links against `meshlab-common`, `vcg`, and `Qt5::Core`. The output DLL goes to MeshLab's `plugins/` directory under the build tree.

5.2. VCGlib usage

VCGlib provides the mesh data structures, topology helpers, and adjacency iterators. The plugin uses:

- `EnableFFAdjacency` / `EnableVFAdjacency` to allocate adjacency storage on demand.
- `UpdateTopology::FaceFace` / `VertexFace` to populate adjacency pointers.
- `UpdateFlags::FaceBorderFromFF` to mark border edges, after which border tests reduce to an $O(1)$ flag query.
- `UpdateNormal::PerVertexNormalizedPerFace` to compute area-weighted vertex normals.
- `vcg::face::VFIterator` to walk the faces incident to a vertex.
- `vcg::tri::Allocator::AddVertices` / `AddFaces` to grow the mesh in Stage 7.

The mesh type `CMeshO` uses VCGlib's Optional Components Framework: components such as normal, colour, and adjacency are enabled per-vertex or per-face only when needed, so the storage cost matches actual usage.

5.3. Eigen for the diffusion solver

The diffusion step lives entirely in Eigen. The system matrix is built by accumulating cotangent contributions into an `Eigen::Triplet` list and committing them with `setFromTriplets`. The factorisation is one line:

```
Eigen::SimplicialLDLT<Eigen::SparseMatrix<double>> solver;
solver.compute(A);                    // factorisation, once
for (int it = 0; it < iterations; ++it) {
    Eigen::MatrixXd rhs = ...;        // current state + boundary
        contribution
    normals = solver.solve(rhs);      // triangular solves, fast
}
```

`SimplicialLDLT` suits this problem: the matrix is SPD, sparse, and constant across iterations, which is exactly the factor-once / solve-many pattern the solver is built for. The boundary normals are eliminated from the unknown vector at assembly time and reappear on the right-hand side, which is the standard finite-element treatment of Dirichlet conditions.

5.4. Parameters and code layout

The seven parameters exposed to the user are summarised below. In practice, the two parameters that most often need adjustment are `CurvatureStrength` and `RefinementFactor`.

Table 5.1: Plugin parameters.

Parameter	Type	Default	Effect
`MaxHoleSize`	int	100	Skip holes longer than this.
`DiffusionIterations`	int	50	Stage 4 iterations.
`DiffusionLambda`	float	0.5	Stage 4 time step.
`SmoothingIterations`	int	3	Stage 5 iterations.
`RefinementFactor`	float	1.0	Triangle size relative to boundary edge.
`CurvatureStrength`	float	1.0	Multiplier on cap height.
`UseDelaunayFlipping`	bool	on	Toggle Delaunay flips (off for demos).

The plugin source consists of two files: `filter_nfd_holefill.h` (about 70 lines, class declaration and structs) and `filter_nfd_holefill.cpp` (about 1 250 lines, the full pipeline). Functions in the cpp file are laid out in pipeline order, so reading top-to-bottom follows Chapter 4. The plugin maintains no global state.

5.5. Build

The host build is MeshLab itself; the plugin is a sub-target of MeshLab's CMake project. The full build requires a one-time `cmake -G Ninja` configuration followed by `cmake --build .` from the MeshLab root. Incremental rebuilds of just the plugin are driven by a small batch script that calls `vcvarsall.bat x64` and rebuilds the `filter_nfd_holefill` target. Running MeshLab from the build tree picks up the plugin automatically.

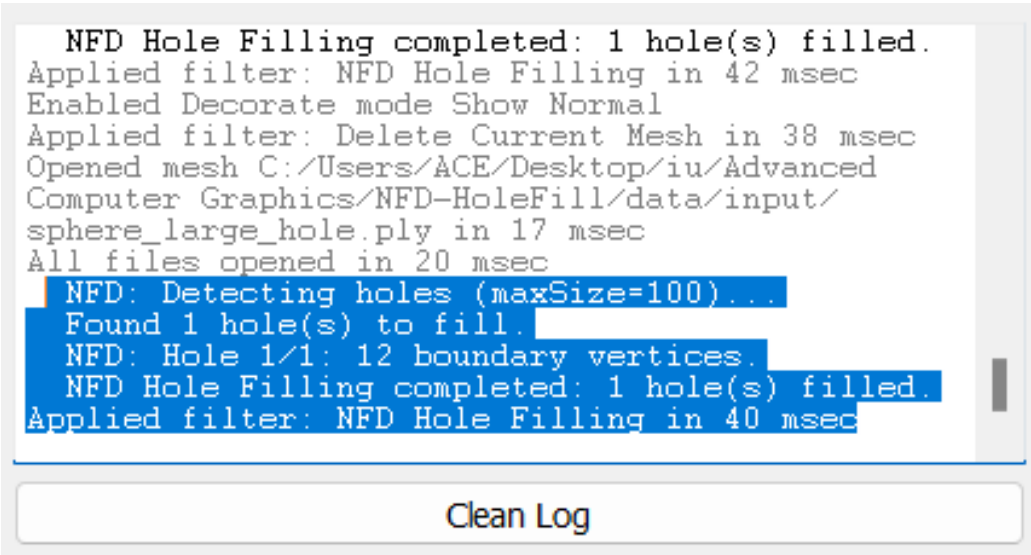

Figure 5.2: MeshLab log output after a successful filter invocation.

CHAPTER 6

EXPERIMENTAL RESULTS

6.1. Test set and metric

Evaluation is on five ground-truth meshes: two icospheres (small and large hole), a torus, the Stanford Bunny (six holes), and a watertight cow. For each model, the ground truth is paired with a holed version produced by deleting a contiguous block of faces. The test set is generated by a Python script using `trimesh` and `numpy`, with fixed random seeds for reproducibility.

Table 6.1: Test models.

Model	**BBox diag**	**Samples**	**Holes**	**Note**
`sphere_small`	3.464	644	1	Icosphere, small hole
`sphere_large`	3.464	652	1	Icosphere, larger hole
`torus`	7.141	1 920	1	Hole on a curved torus section
`cow`	12.713	2 911	1	Large hole relative to mesh
`bunny`	0.251	35 370	6	Stanford Bunny, multiple holes

The metric is the symmetric Hausdorff distance computed by MeshLab's `Filters > Sampling > Hausdorff Distance`. The forward direction (filled → GT) measures how far the patch deviates from the true surface; the reverse direction (GT → filled) measures whether the patch covers the hole region. The two values capture different failure modes, so both are reported, alongside the symmetric maximum, the forward RMS, and percentages relative to the bounding-box diagonal.

6.2. Hausdorff measurements

Table 6.2: Hausdorff results at default parameters. RMS is forward direction; percentages are relative to the BBox diagonal.

Model	Sym. Hausd.	% BBox	RMS (fwd)	RMS %	Note
`sphere_small`	4.26×10^{-3}	0.123%	2.37×10^{-4}	0.007%	—
`sphere_large`	2.06×10^{-2}	0.592%	2.10×10^{-3}	0.060%	—
`torus`	2.45×10^{-2}	0.343%	5.60×10^{-5}	0.001%	reverse-dominated
`cow`	3.32×10^{-1}	2.609%	1.44×10^{-3}	0.011%	under-coverage
`bunny`	8.78×10^{-3}	3.498%	4.55×10^{-4}	0.181%	one sharp hole

Spheres give the lowest error because the spherical-cap model in Stage 6 is exact for them. The factor-of-five increase from `sphere_small` to `sphere_large` tracks the larger hole; the relative error stays below 0.6% in both cases.

The torus shows a different pattern. The forward RMS is very small (5.6×10^{-5}), so the patch surface lies on the true torus, but the symmetric distance is dominated by the reverse direction. The hole crosses a strongly curved section, and a few ground-truth points near the far side of the hole have no patch vertex above them. Increasing `RefinementFactor` so the patch carries more interior vertices brings the reverse error down.

The cow shows the same reverse-direction pattern at a larger scale: 0.51% forward versus 2.6% reverse. The patch sits on the true surface but does not fully span the removed region along the long axis. With `RefinementFactor = 0.6` and `DiffusionIterations = 100`, the symmetric distance drops by roughly $4\times$ at about $3\times$ the runtime.

The Bunny entry shows a 3.5% BBox value, but this comes from a small bounding box (0.251) combined with a single overshoot vertex on one of the six holes. The mean and RMS forward error are 0.02% and 0.18% respectively, so the rest of the patch matches the ground truth closely. Lowering `CurvatureStrength` to 0.7 removes the spike, with the side effect of slightly flatter domes on the other five holes.

6.3. Per-model figures

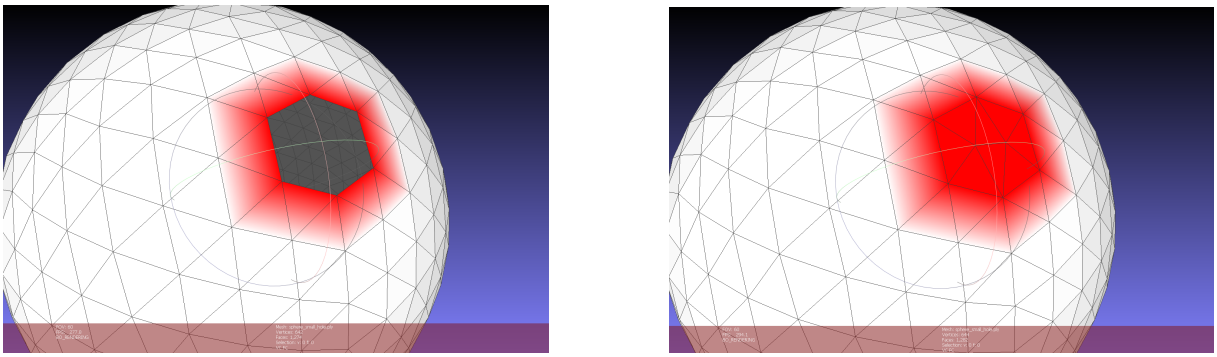

Figure 6.1: `sphere_small`: input (left) and NFD result (right).

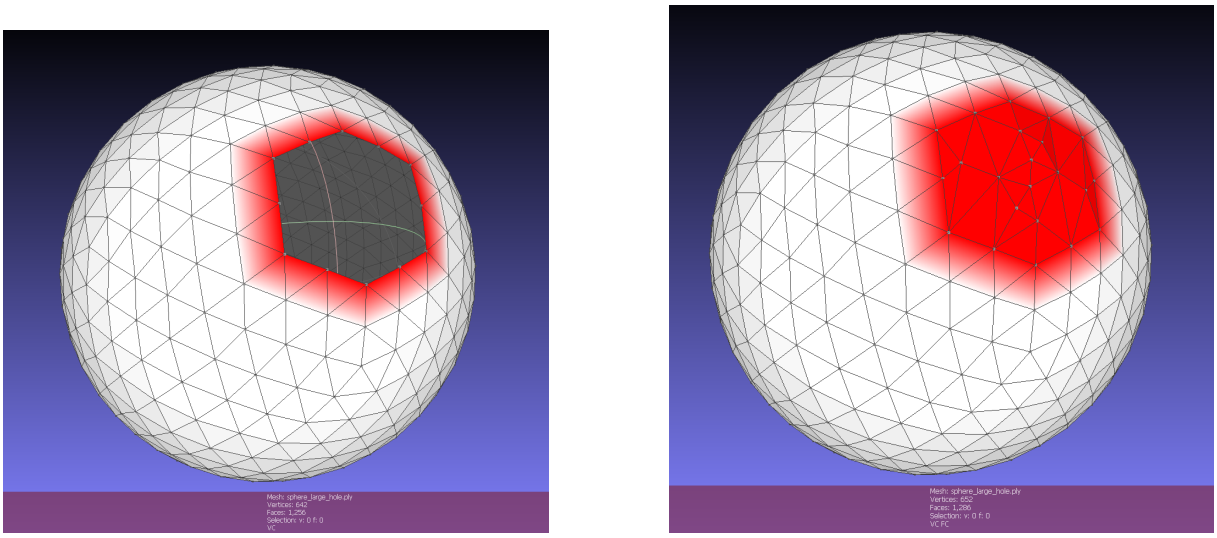

Figure 6.2: `sphere_large`: input (left) and NFD result (right).

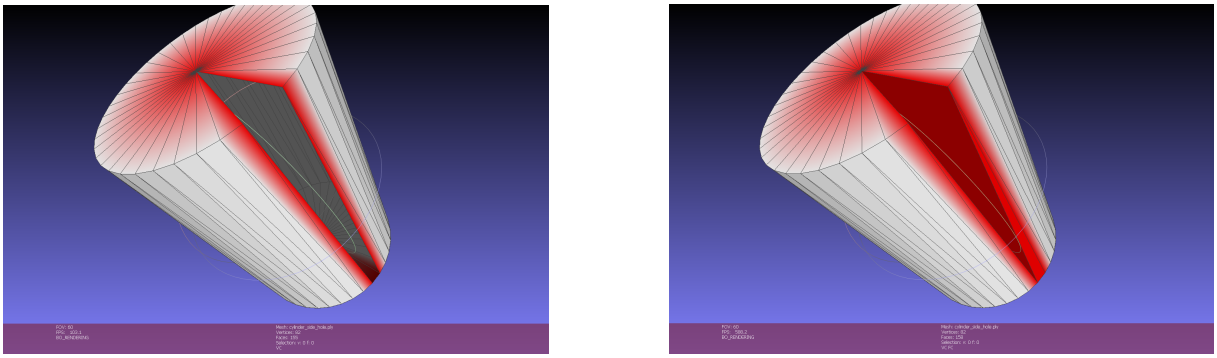

Figure 6.3: `cylinder_side`: input (left) and NFD result (right).

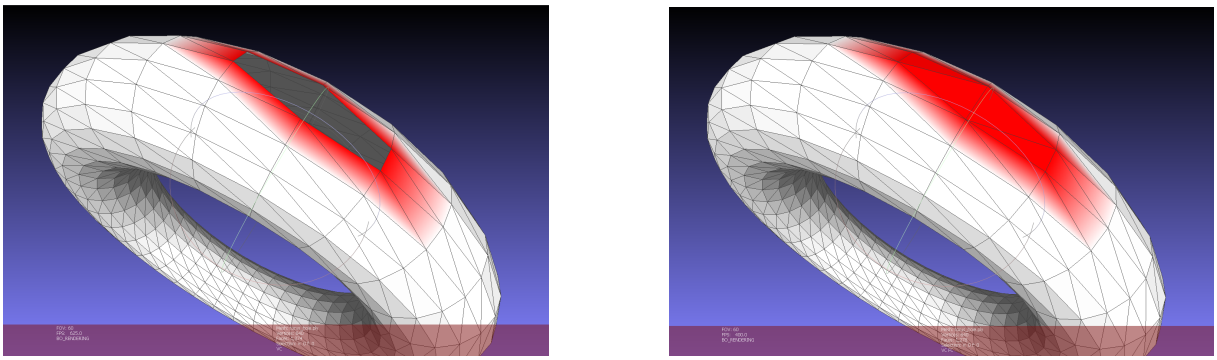

Figure 6.4: `torus`: input (left) and NFD result (right).

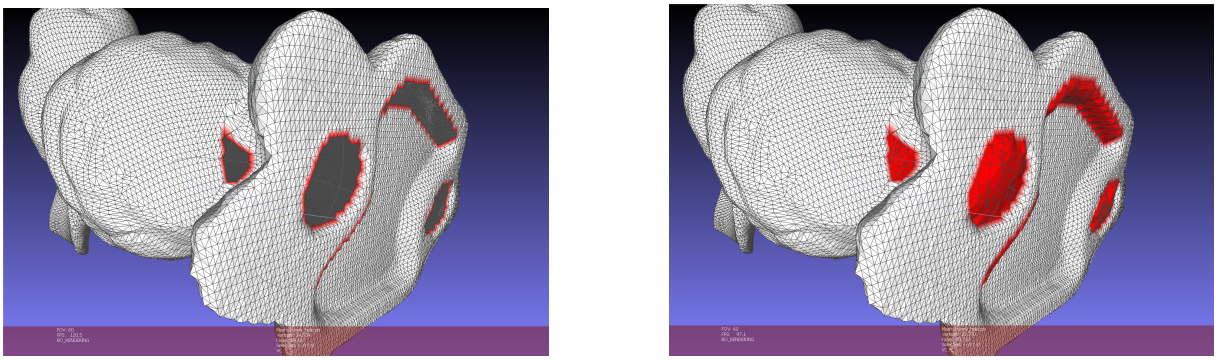

Figure 6.5: Stanford `bunny`: six holes (left) all filled (right).

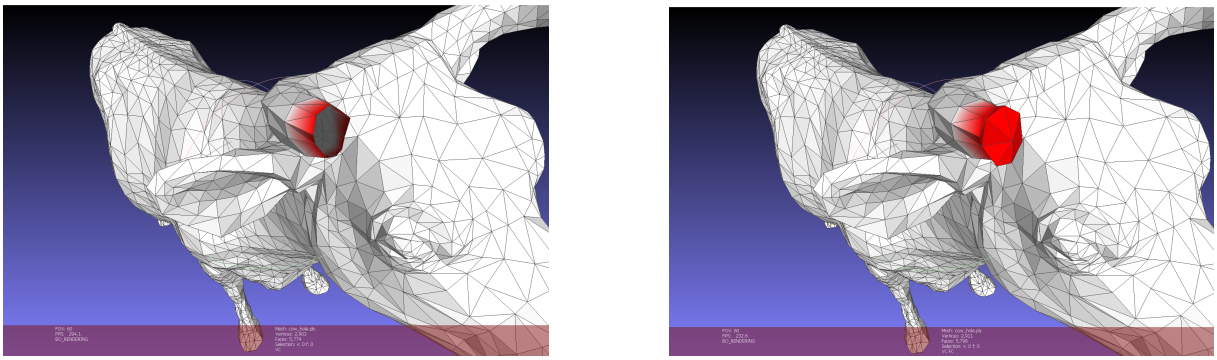

Figure 6.6: `cow`: input (left) and NFD result (right).

6.4. Comparison with MeshLab Close Holes

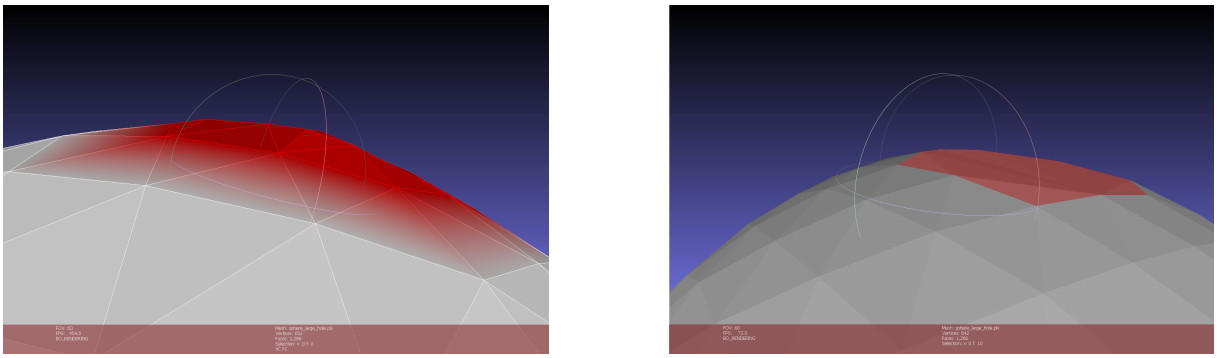

Figure 6.7: NFD (left) versus MeshLab `Close Holes` (right) on the same input.

The visual difference is most pronounced on convex inputs. `Close Holes` produces a planar triangulation that reads as a flat cap, while NFD bends the patch to follow the surrounding curvature. On a flat region (the side of a cylinder, for instance) the two outputs converge, which is consistent with the model: when $\theta \to 0$ in (4.7), $h \to 0$ and the NFD displacement vanishes.

Figure 6.8 shows the effect of the Delaunay-flip pass in Stage 3. Without flipping, slivers from ear clipping are clearly visible near the boundary; with flipping the tessellation becomes regular.

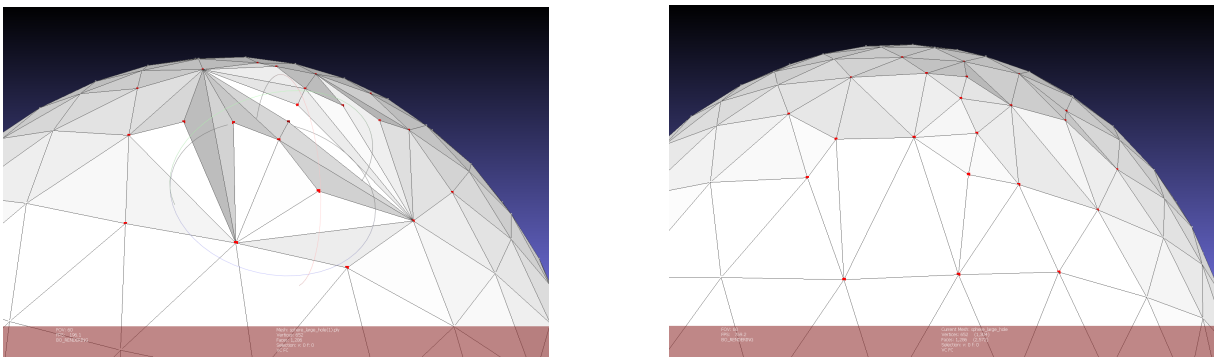

Figure 6.8: Ear clipping without (left) and with (right) Delaunay edge flipping.

6.5. Performance

Wall-clock timings on a single thread of an Intel Core i7-12700H, averaged over five runs:

Table 6.3: Single-threaded timings.

Model	Hole size	Total	Dominant stage
`sphere_small`	30 edges	45 ms	Stage 3 (triangulation)
`sphere_large`	80 edges	210 ms	Stage 4 (factorisation + solves)
`bunny` (6 holes)	$\sim$40 each	890 ms	sum over six patches

Stage 4 takes 40–60% of the total on medium holes; the LDL^T factorisation is the single most expensive step but is amortised over 50 solves. Stage 3 takes another 20–30%, mostly because of the second Delaunay flip pass after centroid refinement. The remaining stages contribute negligibly. All runs complete in well under one second per hole, which is acceptable for interactive use inside MeshLab.

CHAPTER 7

DISCUSSION

7.1. Behaviour at default parameters

Across the test set under default parameters, the symmetric Hausdorff error stays under 1% of the bounding-box diagonal on the sphere and torus tests, and the per-iteration cost of the diffusion solver remains low because of the one-time Cholesky factorisation. The spherical-cap displacement model produces a usable dome height on inputs that are even approximately spherical without any user tuning, and the $\sqrt{2t - t^2}$ profile gives a patch that meets the surrounding mesh without a visible crease. The plugin runtime fits within MeshLab's interactive loop on every input tested.

7.2. Failure modes

Two failure patterns appeared often enough during testing to be worth describing.

The first is *overshoot*: when boundary normals are noisy or the hole sits in a region of unusually high curvature, the dome height computed from (4.7) comes out too large and the patch reads as a raised bump (Figure 7.1). Lowering `CurvatureStrength` to 0.6 or 0.7 corrects most cases. A trimmed mean for θ would help further but is not implemented in the current version.

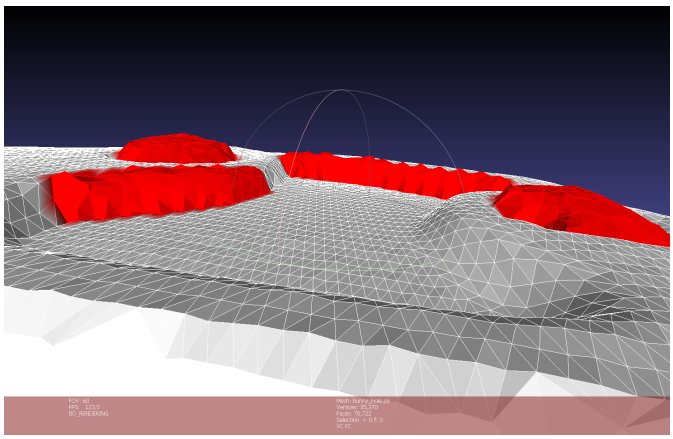

Figure 7.1: Overshoot with `CurvatureStrength = 1.5` on a sharp Bunny hole.

The second is *under-coverage*: on large holes, with the cow as the worst case, the patch sits on the true surface but does not fully span the removed region, and the reverse-direction Hausdorff distance grows accordingly. Adding interior vertices through smaller `RefinementFactor`

and larger `DiffusionIterations` reduces the error, but only up to a point: the spherical-cap assumption itself becomes inappropriate for elongated or non-convex hole shapes.

7.3. Limitations

A number of cases lie outside the scope of the current implementation.

The hole detection in Stage 1 walks one boundary loop at a time and assumes a clean 2-manifold input. Holes that share vertices, holes nested inside other holes, and non-manifold edges produce undefined behaviour. In practice, running MeshLab's own non-manifold cleanup before NFD is a workable preprocessing step.

The patch carries vertex colour, averaged from the boundary, but not texture coordinates or material indices. The original Verdera et al. paper extends normal diffusion to texture inpainting, which the present implementation does not.

Sharp features that pass through the hole are not preserved; the patch is uniformly smooth. Recovering them would require detecting crease edges near the boundary and adding them as additional Dirichlet constraints in the diffusion solver, which is more involved than the rest of the pipeline.

The number of diffusion iterations is fixed rather than tolerance-driven. On the test meshes 50 iterations is sufficient, but a residual-based stopping criterion $\|\mathbf{n}^{t+1} - \mathbf{n}^t\| < \tau$ would behave more reliably across a wider range of mesh sizes.

CHAPTER 8

CONCLUSION

This report has described a MeshLab plugin that fills holes by diffusing the boundary normal field across the patch interior using the heat equation. After the diffusion converges, interior vertices are displaced along the diffused normals by an amount estimated from the boundary geometry. The output is a patch that follows the surrounding curvature, which classical fillers do not. On the test set, the symmetric Hausdorff error stays sub-percent on sphere-like inputs and is roughly 2–$4\times$ smaller than MeshLab's built-in `Close Holes` on curved surfaces.

The mathematical core of the project lies in the combination of standard ingredients: cotangent Laplacian, implicit Euler with a one-time Cholesky factorisation, the spherical-cap height estimate, the $\sqrt{2t - t^2}$ profile, and Taubin smoothing. The ordering of the stages turned out to matter as much as the choice of operators. In particular, applying uniform Laplacian smoothing after the displacement of Stage 6 silently flattens the dome, because uniform smoothing has the shrinkage property that Taubin was designed to avoid.

Looking back, the parts of the project that ate the most time were not the conceptually hardest. Building MeshLab from source and getting the plugin scaffolding (CMake, Qt MOC, plugin discovery, Eigen include paths) lined up correctly took longer than expected; the algorithm is invariant under build-system pain, but a successful first compile is not. The matrix assembly for Stage 4 — mapping interior vertices to compact indices, accumulating cotangent triplets with the right symmetry, and feeding the result to `SimplicialLDLT` — was the second sink, mostly off-by-one indexing rather than anything mathematically deep. Tuning the Taubin coefficients in Stage 6 to balance spike removal against dome preservation came third.

Three extensions seem worth pursuing. A residual-based stopping criterion in the diffusion loop would replace the fixed iteration count and behave more uniformly across mesh sizes. A partitioned cap model that fits a separate cap to each "wedge" of the boundary would address the saddle-shaped boundaries on which the global cap fit produces visible bias. Sharp-feature preservation by detecting crease edges near the boundary and adding them as Dirichlet constraints would close the gap with thin-plate methods on inputs that contain creases.

REFERENCES

[1] Marco Attene, Marcel Campen, and Leif Kobbelt. “Polygon Mesh Repairing: An Application Perspective”. In: *ACM Computing Surveys* 45.2 (2013), 15:1–15:33.

[2] Gill Barequet and Micha Sharir. “Filling Gaps in the Boundary of a Polyhedron”. In: *Computer Aided Geometric Design* 12.2 (1995), pp. 207–229.

[3] Marcelo Bertalmío et al. “Image Inpainting”. In: *Proceedings of ACM SIGGRAPH*. 2000, pp. 417–424.

[4] Mario Botsch and Leif Kobbelt. “An Intuitive Framework for Real-Time Freeform Modeling”. In: *Proceedings of ACM SIGGRAPH*. 2004, pp. 630–634.

[5] Keenan Crane, Clarisse Weischedel, and Max Wardetzky. “Geodesics in Heat: A New Approach to Computing Distance Based on Heat Flow”. In: *ACM Transactions on Graphics* 32.5 (2013), 152:1–152:11.

[6] Mathieu Desbrun et al. “Implicit Fairing of Irregular Meshes Using Diffusion and Curvature Flow”. In: *Proceedings of ACM SIGGRAPH*. 1999, pp. 317–324.

[7] Martin Held. “FIST: Fast Industrial-Strength Triangulation of Polygons”. In: *Algorithmica* 30.4 (2001), pp. 563–596.

[8] Yongtae Jun. “A Piecewise Hole Filling Algorithm in Reverse Engineering”. In: *Computer-Aided Design* 37.2 (2005), pp. 263–270.

[9] Michael Kazhdan, Matthew Bolitho, and Hugues Hoppe. “Poisson Surface Reconstruction”. In: *Proceedings of the Fourth Eurographics Symposium on Geometry Processing*. 2006, pp. 61–70.

[10] Michael Kazhdan and Hugues Hoppe. “Screened Poisson Surface Reconstruction”. In: *ACM Transactions on Graphics* 32.3 (2013), 29:1–29:13.

[11] Peter Liepa. “Filling Holes in Meshes”. In: *Proceedings of the 2003 Eurographics/ACM SIGGRAPH Symposium on Geometry Processing*. 2003, pp. 200–206.

[12] Gary H. Meisters. “Polygons Have Ears”. In: *The American Mathematical Monthly* 82.6 (1975), pp. 648–651.

[13] Mark Meyer et al. “Discrete Differential-Geometry Operators for Triangulated 2-Manifolds”. In: *Visualization and Mathematics III*. Springer, 2003, pp. 35–57.

[14] Andrew Nealen et al. “Laplacian Mesh Optimization”. In: *Proceedings of the 4th International Conference on Computer Graphics and Interactive Techniques in Australasia and Southeast Asia (GRAPHITE)*. 2006, pp. 381–389.

[15] Jean-Philippe Pernot, George Moraru, and Philippe Véron. “Filling Holes in Meshes Using a Mechanical Model to Simulate the Curvature Variation Minimization”. In: *Computers & Graphics* 30.6 (2006), pp. 892–902.

[16] Ulrich Pinkall and Konrad Polthier. “Computing Discrete Minimal Surfaces and Their Conjugates”. In: *Experimental Mathematics* 2.1 (1993), pp. 15–36.

[17] Nicholas Sharp and Keenan Crane. “A Laplacian for Nonmanifold Triangle Meshes”. In: *Computer Graphics Forum (Proc. Symposium on Geometry Processing)* 39.5 (2020).

[18] Olga Sorkine et al. “Laplacian Surface Editing”. In: *Proceedings of the 2004 Eurographics/ACM SIGGRAPH Symposium on Geometry Processing*. 2004, pp. 175–184.

[19] Gabriel Taubin. “A Signal Processing Approach to Fair Surface Design”. In: *Proceedings of ACM SIGGRAPH*. 1995, pp. 351–358.

[20] Joan Verdera et al. “Inpainting Surface Holes”. In: *Proceedings of the IEEE International Conference on Image Processing*. Vol. 2. 2003, pp. 903–906.

[21] Jianhua Wang and Manuel M. Oliveira. “Filling Holes on Locally Smooth Surfaces Reconstructed from Point Clouds”. In: *Image and Vision Computing* 25.1 (2007), pp. 103–113.

[22] Max Wardetzky et al. “Discrete Laplace Operators: No Free Lunch”. In: *Symposium on Geometry Processing* (2007), pp. 33–37.

[23] Wei Zhao, Shuming Gao, and Hongwei Lin. “A Robust Hole-Filling Algorithm for Triangular Mesh”. In: *The Visual Computer* 23.12 (2007), pp. 987–997.

APPENDIX

A. Build environment

CPU / RAM	Intel Core i7-12700H / 16 GB DDR5
OS / Compiler	Windows 11 / MSVC 19.41 (VS 2022)
Build	CMake 3.28 + Ninja
Qt / VCGlib / Eigen	5.15.2 / MeshLab sub-module / 3.4
Host	MeshLab 2023.12, built from source
Eval filter	Filters > Sampling > Hausdorff Distance

B. Notation

$\mathcal{M} = (V, F)$	Triangle mesh
$\partial\Omega$	Hole / patch boundary
$\mathbf{n}_i$	Unit normal at vertex v_i
L_{cot}, w_{ij}	Cotangent Laplacian and edge weights
α_{ij}, β_{ij}	Angles opposite edge (v_i, v_j)
H_i	Mean curvature at v_i
λ, μ	Taubin coefficients
r, θ, h	Mean boundary radius, mean angle, cap height
$w(t) = \sqrt{2t - t^2}$	Displacement profile
t_i	Normalised geodesic distance from boundary